A Study of Optimization Methods for Full Fine-Tuning of Qwen2.5-0.5B

Odilzhon Negmatuloev

1 Introduction

This work presents a study of optimization methods for full fine-tuning of a small language model (SLM), Qwen2.5-0.5B. The study compares three distinct optimization approaches: the widely adopted adaptive optimizer AdamW, the recently proposed matrix-aware optimizer Muon, and the zeroth-order optimizer MeZO.

All experiments are conducted in the regime of full fine-tuning, where all model parameters are updated. This investigation addresses the following research questions:

1. How do AdamW, Muon, and MeZO compare in terms of convergence speed and final training loss?
2. What are the memory and computational trade-offs of each optimizer?
3. Do differences in training dynamics translate to differences in downstream task performance?

2 Optimization Methods

2.1 AdamW

AdamW [1] extends the Adam optimizer by *decoupling* weight decay from the adaptive gradient update. At each training step t, given gradient $g_t = \nabla_\theta \mathcal{L}(\theta_t)$, the first and second moment estimates are updated as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{3}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda\theta_t \tag{5}$$

where η is the learning rate, λ is the weight decay coefficient, β_1 and β_2 are exponential decay rates, and ϵ is a small constant for numerical stability.

Unlike classical L_2 regularization applied inside the adaptive update, AdamW applies weight decay as a separate multiplicative shrinkage step, ensuring that regularization strength is independent of the adaptive scaling of gradients.

2.2 Muon

Muon (Momentum Orthogonalized by Newton-Schulz) [2] is a matrix-aware optimizer that applies orthogonalization-based updates to 2D weight matrices while using standard adaptive methods for other parameters. The core innovation lies in preserving spectral properties of weight matrices during optimization.

For matrix parameters $W \in R^{m\times n}$, Muon computes:

$$W_{t+1} = \text{NewtonSchulz}(W_t - \eta G_t) \tag{6}$$

where G_t is the momentum-accumulated gradient and NewtonSchulz$(\cdot)$ denotes the Newton-Schulz orthogonalization procedure. This operation approximately orthogonalizes the update using a Newton–Schulz iteration, helping preserve spectral properties of weight matrices rather than performing an exact projection onto the nearest orthogonal matrix.

2.3 MeZO

MeZO (Memory-efficient Zeroth-Order Optimization) [3] is based on the classical Simultaneous Perturbation Stochastic Approximation (SPSA) gradient estimator. Unlike first-order methods, MeZO does not rely on backpropagation and instead estimates gradients using forward evaluations only.

Given parameters $\theta \in R^d$ and a minibatch $\mathcal{B}$, a random perturbation vector $z \sim \mathcal{N}(0, I_d)$ is sampled. The SPSA gradient estimator is:

$$\hat{\nabla}\mathcal{L}(\theta;\mathcal{B}) = \frac{\mathcal{L}(\theta + \epsilon z;\mathcal{B}) - \mathcal{L}(\theta - \epsilon z;\mathcal{B})}{2\epsilon}\, z \tag{7}$$

where ϵ is a small perturbation scale. The parameter update is then given by:

$$\theta_{t+1} = \theta_t - \eta_t \hat{\nabla}\mathcal{L}(\theta_t;\mathcal{B}_t). \tag{8}$$

Each SPSA estimate requires two forward passes through the model. Unlike standard zeroth-order implementations, which explicitly store the perturbation vector z, MeZO uses an in-place implementation that reconstructs z via a fixed random seed. This allows the method to achieve a memory footprint equivalent to inference-time memory, since no gradient tensors or optimizer states are stored.

While MeZO eliminates backward propagation and gradient storage, the gradient estimate has higher variance compared to first-order methods, which may negatively impact convergence stability.

3 Experimental Setup

3.1 Computational Infrastructure

Experiments were conducted using the following configuration:

- **Hardware:** 2× NVIDIA Tesla T4 GPUs (15 GB VRAM each)
- **Precision:** FP32 (Tesla T4 lacks native BF16 support)
- **Budget constraint:** 30 GPU hours per week

The memory and compute limitations of the T4 architecture necessitated careful experimental design and motivated dataset size reduction.

3.2 Model Architecture

The base model used is Qwen2.5-0.5B, a 0.5 billion parameter causal language model. All parameters were updated during fine-tuning.

3.3 Dataset

Training was performed on OpenWebText-10k, consisting of the first 10,000 examples from the OpenWebText corpus. This subset was selected to accommodate the computational budget while enabling meaningful optimizer comparison.

Text preprocessing followed standard causal language modeling procedures. Documents were tokenized using the model's native tokenizer, concatenated, and packed into fixed-length sequences of 512 tokens. Sequence packing maximizes GPU utilization by minimizing padding overhead and ensures efficient use of the available context window during training.

3.4 Training Configuration

To ensure fair comparison, core hyperparameters were standardized across first-order methods (AdamW and Muon). MeZO required adjusted hyperparameters due to its fundamentally different optimization dynamics.

Common settings:

- Sequence length: 512 tokens
- Per-device batch size: 2 (gradient accumulation steps: 8; effective batch size: 16)

- Learning rate scheduler: Cosine
- Precision: FP32

AdamW and Muon hyperparameters:

- Learning rate: 5×10^{-5}
- Weight decay: 0.05

MeZO hyperparameters:

- Learning rate: 2×10^{-5}
- Zo-eps: 1×10^{-3}
- Weight decay: 0.0
- Perturbation parameter: $\epsilon = 10^{-3}$

The lower learning rate and zero weight decay for MeZO were determined through preliminary experiments and align with recommendations in the original work.

3.5 Evaluation Methodology

Models were evaluated using the EleutherAI Language Model Evaluation Harness on five commonsense reasoning benchmarks in zero-shot setting:

- **PIQA**: Physical interaction question answering
- **ARC-Easy**: Elementary-level science questions
- **ARC-Challenge**: Complex science questions
- **Winogrande**: Pronoun resolution requiring commonsense reasoning
- **HellaSwag**: Sentence completion with commonsense inference

4 Implementation Details

4.1 Muon Integration

The Muon optimizer was integrated from the Moonlight repository [2] as a standard `torch.optim.Optimizer` subclass. Integration required minimal modification: the optimizer class was incorporated into the codebase and instantiated via the `create_optimizer` method of Hugging Face's `Trainer`.

Two Muon-based configurations were evaluated. In the **pure Muon** setting, Muon was applied to all 2D weight matrices (229,638,144 parameters, 59.60% of total parameters), while the remaining parameters were left unchanged. In the

hybrid Muon+AdamW configuration, Muon was applied to the same subset of matrix parameters (59.60%), and the remaining 155,644,928 parameters (40.40%) were optimized using AdamW.

The original implementation includes the following caveats:

- "We believe it may not work well for finetuning pretrained models."
- "We believe this optimizer is unlikely to work well with small batch sizes."

4.2 MeZO Implementation

The MeZO implementation was adapted from the original repository [3]. Due to dependency conflicts and deprecated API usage in the original codebase, core algorithmic components were extracted and reimplemented within a modern `transformers.Trainer` framework.

A custom `MeZOTrainer` class was developed, inheriting from `transformers.Trainer` and overriding the training loop. Key functions—`zo_step` (gradient estimation), `zo_update` (parameter update), and `_perturb_parameters`—were ported with minimal modifications to ensure algorithmic fidelity while maintaining compatibility with current library versions.

5 Results

5.1 Training Dynamics

Figure 1 presents training loss trajectories, and Table 1 summarizes key statistics.

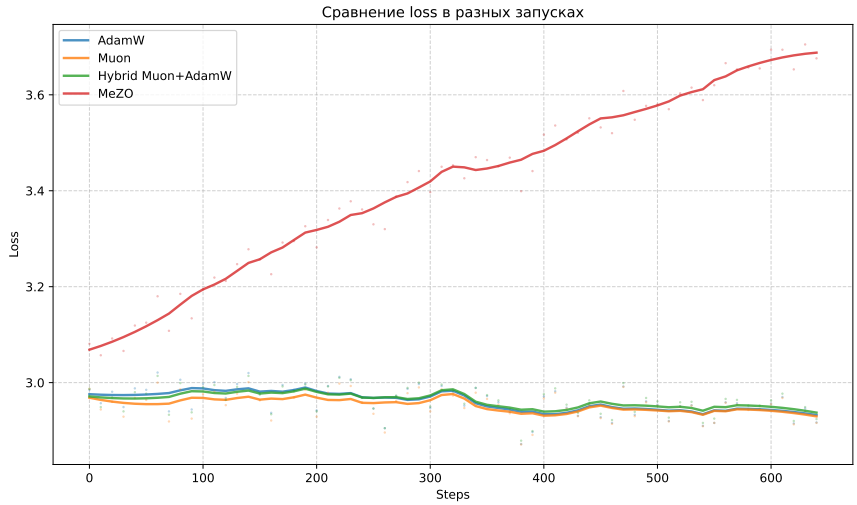

Figure 1: Training loss curves across optimizers. AdamW and Muon exhibit nearly identical convergence trajectories, while MeZO shows substantially higher loss and increased instability.

Optimizer	**Initial**	**Min**	**Max**	**Final**
AdamW	2.985	2.872	3.020	2.916
Muon	2.987	2.871	3.000	2.915
Hybrid Muon+AdamW	2.987	2.878	3.014	2.923
MeZO	3.080	3.057	3.705	3.675

Table 1: Training loss statistics. Best values in bold.

Key observations:

- AdamW, Muon, Hybrid Muon+AdamW demonstrate nearly identical convergence behavior
- MeZO exhibits substantially degraded performance: final loss 26% higher than AdamW, with pronounced instability (max loss 3.705 vs 3.020 for AdamW).

5.2 Computational Efficiency

Table 2 presents training efficiency metrics.

- **Muon** reduces peak memory usage by approximately 12.0% relative to AdamW (7.660 GB vs 8.705 GB), reflecting its reduced optimizer state footprint for matrix parameters.

Optimizer	Final Loss	Time (hrs)	Peak Memory (GB)
AdamW	2.916	2.5	8.705
Muon	2.915	3.2	7.660
Hybrid Muon+AdamW	2.923	3	8.240
MeZO	3.675	1.7	7.479

Table 2: Training efficiency comparison. Peak memory refers to GPU_0 maximum usage.

Model	PIQA	ARC-E	ARC-C	Winogrande	HellaSwag	Avg
Base (Qwen2.5-0.5B) –	0.7051	0.6465	0.2910	0.5643	0.4059	0.5226
AdamW	0.7035	0.6667	0.2961	0.5683	0.3982	0.5266
Muon	0.7029	0.6654	0.2909	0.5674	0.3989	0.5251
Muon+AdamW	0.7013	0.6671	0.3003	0.5596	0.4006	0.5258
MeZO	0.6382	0.5370	0.2619	0.5351	0.3592	0.4663

Table 3: Zero-shot accuracy on downstream tasks. Average (Avg) is computed across the five benchmarks.

- **Hybrid Muon+AdamW** achieves a smaller memory reduction of approximately 5.3% (8.240 GB vs 8.705 GB), consistent with only 59.6% of parameters being optimized with Muon.
- **MeZO** achieves the lowest memory footprint (7.479 GB), corresponding to a 14.1% reduction relative to AdamW. This aligns with its gradient-free design, which avoids storing first- and second-moment optimizer states.

5.3 Downstream Task Performance

Table 3 presents zero-shot accuracy on five commonsense reasoning benchmarks. In addition to per-task results, we report the average accuracy across tasks and the difference relative to the base pretrained model.

Key findings:

- AdamW, Muon, Hybrid Muon+AdamW achieve nearly identical average performance (0.5266 vs 0.5251), with absolute differences below 0.002 accuracy, indicating practical equivalence under the current setup.
- Compared to the base pretrained model (0.5226 average accuracy), fine-tuning with AdamW and Muon yields a very small improvement (+0.002 to +0.004). Given the reported standard errors of individual tasks (0.005–0.014), these differences fall within expected statistical variation and should not be interpreted as meaningful gains.

- MeZO underperforms substantially, showing a 5.6 percentage point drop relative to the base model, consistent across all tasks and aligned with its degraded training convergence.

Downstream differences among AdamW, Muon, and Hybrid Muon+AdamW remain negligible. While MeZO exhibits clear degradation. The marginal improvements over the base model suggest that short-horizon full fine-tuning on a 10k subset provides limited task-level benefit in this regime.

6 Discussion

6.1 AdamW vs Muon: Comparable Performance with Reduced Memory

The experimental results indicate that Muon achieves practical performance parity with AdamW in both training dynamics and downstream benchmark accuracy. Final training losses differ by less than 0.001, and average downstream accuracy differs by less than 0.002 absolute points—well within the reported task-level standard errors.

In terms of memory efficiency, the pure Muon configuration reduces peak GPU memory usage by approximately 12% relative to AdamW (7.66 GB vs 8.71 GB). The hybrid Muon+AdamW configuration yields a smaller reduction of approximately 5.3%, consistent with only 59.6% of parameters being optimized using Muon.

Overall, within the current small-scale fine-tuning regime, Muon provides measurable memory savings without observable degradation in optimization stability or downstream performance. However, given the limited dataset size and single-epoch training setup, these results should be interpreted as evidence of equivalence under constrained conditions rather than proof of general optimizer superiority.

6.2 MeZO: Memory Efficiency at the Cost of Convergence Quality

MeZO delivers on its promise of reduced memory consumption (14.1% reduction), making it potentially valuable for extremely memory-constrained scenarios. However, this comes at substantial cost:

- **Convergence degradation:** Final training loss 26% higher than first-order methods
- **Downstream performance gap:** 11.4% lower average accuracy than AdamW

6.3 Limitations

Several experimental constraints limit the generalizability of these findings:

- **Dataset size:** The 10k-example subset may not reflect behavior on full-scale corpora.
- **Training duration:** 1 epoch (658 steps) represents early-stage fine-tuning; longer training may reveal different optimizer characteristics.
- **Hardware constraints:** FP32 precision on T4 GPUs differs from typical BF16/FP16 training on modern accelerators.

7 Conclusion

This study provides a controlled empirical comparison of three optimization methods for full fine-tuning of the Qwen2.5-0.5B language model. The key findings are:

1. **Convergence Speed and Final Training Loss.** AdamW and pure Muon exhibit nearly identical convergence dynamics, with final training losses differing by less than 0.001. The hybrid Muon+AdamW configuration shows similarly close behavior. Within the single-epoch, small-batch regime considered here, no meaningful difference in optimization quality can be established between first-order methods. In contrast, MeZO demonstrates substantially degraded convergence, with approximately 26% higher final loss and increased instability.
2. **Memory and Computational Trade-offs.** Pure Muon reduces peak GPU memory usage by approximately 12% relative to AdamW, while the hybrid Muon+AdamW configuration yields a smaller reduction of approximately 5.3%. MeZO achieves the largest memory reduction (14.1%), reflecting its gradient-free design. However, this gain comes at the cost of significantly degraded optimization performance, illustrating a clear trade-off between memory efficiency and convergence reliability in the evaluated regime.
3. **Downstream Task Performance.** AdamW and Muon achieve nearly identical downstream benchmark accuracy, with average differences below 0.002 absolute points—well within reported task-level standard errors. Fine-tuning yields only marginal improvements over the base pretrained model, suggesting limited task-level adaptation in this short-horizon setting. In contrast, MeZO exhibits a consistent and substantial performance drop across benchmarks.

Overall, within the constraints of the current experimental setup, AdamW remains the most reliable choice, with Muon serving as a promising memory-efficient alternative that maintains comparable performance. The substantial memory savings achieved by MeZO come at too high a cost in optimization quality, making it impractical for standard fine-tuning scenarios. At the same time, a fair assessment of its capabilities requires reproducing the experimental

conditions described in the original paper, as the present configuration may not reflect the regime in which MeZO demonstrates its claimed advantages.

Further research with larger batch sizes and more extensive datasets is necessary to better understand the true potential and limitations of both Muon and MeZO.

References

[1] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[2] MoonshotAI. Moonlight: Muon Optimizer for Scalable LLM Training. GitHub repository, 2025. `https://github.com/MoonshotAI/Moonlight`

[3] S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. `https://github.com/princeton-nlp/MeZO`